Progetto di Ingegneria del Software 2025/26

Università Ca' Foscari Venezia

D4: Piano di Testing

Versione: 2.0

Gruppo: DSH

26/01/2026

Document Informations

AlertUp	AU
Deliverable	Piano di Testing
Data di Consegna	26/01/2026
Team Leader	Samuele Zanutto 890913@stud.unive.it
Team members	Davide Lago 895388@stud.unive.it Hamza Rofi 903405@stud.unive.it

Document History

Version	Issue Date	Stage	Changes	Contributors
1.0	18 / 11 / 2025	Draft	Definizione del documento di Piano di Testing con la definizione della modalità di verifica dei requisiti, dei casi di test, dello scheduling del testing, della procedura di registrazione dei test, dei requisiti hardware e software, dei vincoli che condizionano il testing e di altri test.	Samuele Zanutto, Davide Lago, Hamza Rofi.
2.0	26/01/2026	Final	Allineamento finale della documentazione: aggiornati esiti test e allineate date effettive dei test. Revisione generale e correzione dettagli.	Samuele Zanutto, Davide Lago, Hamza Rofi.

Indice

1. Introduzione 5
1.1. Overview generale 5
1.2. Struttura del documento 5
2. Glossario 6
3. Tecniche di testing 7
4. Modalità di verifica dei requisiti 8
4.1. Verifica statica 8
4.2. Verifica dinamica 9
5. Casi di test 10
6. Schedule del testing 17
6.1. Pianificazione dei test 17
6.2. Criteri di test e requisiti associati 18
7. Procedure di registrazione dei test 19
7.1. Monitoraggio dei test 19
7.2. Aggiornamento dei test 20
7.3. Risoluzione dei problemi 21
7.4. Requisiti di tracciabilità, archiviazione e reporting 21
7.5. Automazione e test continui 22
7.6. Archiviazione e retention 22
8. Requisiti hardware e software 23
8.1. Requisiti hardware 23
8.2. Requisiti software 24
8.3. Configurazione dell'ambiente di Test 24
8.4. Compatibilità e Scalabilità 25
8.5. Considerazioni di sicurezza e sostenibilità 25
9. Vincoli che condizionano il testing 25
9.1. Vincoli tecnici 25
9.2. Vincoli organizzativi 25
9.3. Vincoli temporali 26

9.4. Vincoli ambientali e logistici 26
9.5. Vincoli normativi e di sicurezza 26
10. Riferimenti 26

1. Introduzione

1.1. Overview generale

Il presente documento definisce il Piano di Testing per l'applicazione *AlertUp*. L'obiettivo del documento è descrivere in modo completo e sistematico le tecniche, le modalità e le procedure di test adottate per verificare la conformità del sistema ai requisiti funzionali e non funzionali specificati nel Documento dei Requisiti.
Il piano di testing copre sia la componente software (frontend Angular convertito per Android e backend MongoDB) sia la componente hardware, intesa come l'insieme dei dispositivi e sensori coinvolti nel funzionamento dell'app (in particolare GPS e connessione dati).
Vengono introdotte le tecniche di testing selezionate dal team, le modalità di verifica dei requisiti, i casi di test con i rispettivi risultati attesi, la schedule dei test in funzione delle milestone di progetto, le procedure di registrazione e aggiornamento, nonché i requisiti hardware/software necessari all'esecuzione dei test e i vincoli che potrebbero condizionarne l'attività.
Il documento rappresenta inoltre una base operativa per garantire la qualità del prodotto e l'affidabilità del sistema prima della consegna finale.

1.2. Struttura del documento

Il documento è articolato nei seguenti capitoli:

- **Capitolo 1 – Introduzione**: illustra gli obiettivi, la struttura e il contenuto generale del piano di testing.
- **Capitolo 2 – Glossario**: contiene i termini e le abbreviazioni utilizzate nel documento.
- **Capitolo 3 – Tecniche di Testing**: descrive le metodologie di verifica e validazione applicate al progetto.
- **Capitolo 4 – Modalità di Verifica dei Requisiti**: spiega come ciascun requisito verrà verificato e validato.
- **Capitolo 5 – Casi di Test**: elenca e descrive i test specifici per le funzionalità principali del sistema.
- **Capitolo 6 – Schedule del Testing**: definisce la pianificazione temporale delle attività di test.
- **Capitolo 7 – Procedure di Registrazione dei Test**: illustra le modalità di documentazione, monitoraggio, aggiornamento e risoluzione dei problemi.
- **Capitolo 8 – Requisiti Hardware e Software**: specifica l'ambiente di test e le risorse necessarie.
- **Capitolo 9 – Vincoli che Condizionano il Testing**: analizza i fattori tecnici o organizzativi che possono influire sulle attività di verifica.
- **Capitolo 10 – Riferimenti**: elenca i materiali, i documenti e le fonti utilizzati per la redazione del piano di testing.

2. Glossario

Termine	Definizione	Fonte
Verifica statica	Tecnica di verifica che analizza artefatti senza eseguire il codice, ad esempio tramite review, inspection o analisi del codice.	ISTQB Glossary – static testing (glossary.istqb.org)
Verifica dinamica	Tecnica di verifica che esegue il software perché il comportamento venga osservato in esecuzione reale.	LambdaTest – Dynamic Testing (LambdaTest)
Black-box testing	Tecnica in cui il tester analizza il sistema come "scatola nera", concentrandosi sugli input e output senza conoscere la struttura interna.	Opkey – Black-Box Testing (Opkey)
White-box testing	Tecnica di testing che analizza la struttura interna del software, flussi, condizioni e codice sorgente.	ISTQB Glossary – white box testing (glossary.istqb.org)
Back-to-back testing	Tecnica che consiste nell'eseguire due versioni del software con gli stessi input e confrontare gli output per rilevare differenze e regressioni.	LambdaTest – Back-to-back testing (LambdaTest)
Test di integrazione	Tipo di test in cui diversi moduli o componenti del sistema vengono combinati e verificati insieme per validare le interazioni.	ISTQB Glossary – integration testing (erikvanveenendaal.nl)
Test di unità (Unit Testing)	Test che verificano il comportamento di singole unità o moduli del software in isolamento.	LambdaTest – Unit Testing (LambdaTest)
Sessione di inspection	Processo formale di review del codice o degli artefatti che include planning, use-checklist, reading, reporting di difetti.	ISTQB Glossary – inspection (software testing) (ISTQB Official Registration)
Smoke testing	Test preliminare che verifica se le funzioni principali di un build funzionano abbastanza bene da giustificare test ulteriori.	Wikipedia – Smoke testing (software) (Wikipedia)
Test di usabilità (Usability Testing)	Verifica della facilità d'uso, comprensibilità e soddisfazione dell'utente nell'interazione con il sistema.	LambdaTest – Usability Testing (LambdaTest)
Test di regressione	Re-esecuzione dei test su versioni modificate del software per garantire che non siano state introdotte nuove anomalie.	Opkey – Regression Testing (Opkey)

Interoperabilità	Capacità del software di interagire correttamente con altri sistemi e componenti.	ISTQB Glossary – interoperability (ISTQB Official Registration)
Scalabilità	Capacità del sistema di gestire un aumento del carico (es. utenti, transazioni) senza degradazione delle prestazioni.	ISTQB Glossary – scalability (erikvanveenendaal.nl)
Disponibilità (Availability)	Misura della porzione di tempo in cui un sistema è operativo e accessibile per l'uso previsto.	ISTQB Glossary – availability (erikvanveenendaal.nl)
Robustezza (Robustness)	Capacità del software di gestire condizioni di errore o input imprevisti senza fallire.	ISTQB Glossary – robustness (ISTQB Official Registration)

3. Tecniche di testing

viene adottata una **strategia di testing incrementale e combinata**, con l'obiettivo di garantire la stabilità del codice, la qualità delle nuove funzionalità e la coerenza con i requisiti definiti.

In particolare, si utilizzeranno tre approcci complementari:

1. **Testing incrementale bottom-up**:
   Questa tecnica prevede di testare inizialmente le singole componenti di base (ad esempio i moduli per la gestione del database o la geolocalizzazione), per poi passare gradualmente ai moduli di livello superiore e infine all'integrazione complessiva del sistema.
   Tale approccio è particolarmente efficace in un contesto **object-oriented** come quello dell'applicazione, poiché consente di validare le classi e i metodi in modo progressivo, isolando più facilmente eventuali malfunzionamenti.

2. **Black-box Testing**:
   Il **black-box testing** si concentra sul comportamento esterno dell'applicazione, analizzando gli input e verificando che gli output corrispondano alle specifiche dei requisiti funzionali.
   Questo tipo di test sarà utilizzato per validare le funzionalità principali dell'app (registrazione, login, creazione emergenze, notifiche, visualizzazione meteo e mappa) senza tener conto del codice sorgente.
   I test verranno condotti anche da membri del gruppo che non hanno sviluppato direttamente la funzione testata o da tester esterni, per simulare il comportamento di un utente reale.

3. **White-box Testing**:
   Il **white-box testing** analizzerà la logica interna del codice, verificando il corretto flusso di esecuzione e la gestione delle eccezioni.
   Verranno utilizzati test unitari mirati per garantire la copertura del codice e identificare casi limite o condizioni anomale.
   Tale approccio sarà applicato principalmente alla parte **backend** (connessione con il database MongoDB, gestione delle API e sicurezza dell'autenticazione).

Infine, verrà adottato un metodo **back-to-back testing**, confrontando versioni successive dell'applicazione con gli stessi input per rilevare eventuali regressioni o variazioni indesiderate negli output.

L'obiettivo complessivo è assicurare che ogni sprint produca un incremento stabile e coerente, mantenendo la qualità e l'affidabilità del sistema in tutte le sue componenti.

4. Modalità di verifica dei requisiti

La verifica dei requisiti si basa su una combinazione di **verifica statica** e **verifica dinamica**, con l'obiettivo di garantire che ogni requisito definito nel *Documento dei Requisiti* venga rispettato e soddisfi pienamente le aspettative degli stakeholder.

La verifica viene effettuata lungo tutto il ciclo di vita del progetto, in particolare nelle fasi di sviluppo, integrazione e testing finale, seguendo un approccio iterativo coerente con il modello Agile adottato.

4.1. Verifica statica

La verifica statica si concentra sull'analisi del codice e dei documenti progettuali senza eseguire il programma. Questo tipo di controllo viene applicato fin dalle prime fasi dello sviluppo per identificare errori di progettazione, incoerenze nei requisiti o deviazioni dagli standard stabiliti.

Le principali attività di verifica statica comprendono:

- **Code Review e Inspection**: ogni componente del gruppo partecipa periodicamente a sessioni di revisione del codice.
  Durante le inspection vengono seguite quattro fasi principali:
  1. **Pianificazione** – individuazione dei moduli da analizzare e dei ruoli dei partecipanti.
  2. **Definizione di una checklist** – elenco di possibili errori sintattici, logici e di conformità ai requisiti.
  3. **Analisi e lettura del codice** – controllo diretto delle sezioni critiche o recentemente modificate.
  4. **Correzione e documentazione** – revisione dei difetti e aggiornamento delle versioni corrette.

Il team di ispezione sarà composto da:

- L'autore del codice.
- Un lettore che espone il funzionamento durante la sessione.
- Due ispettori incaricati di identificare gli errori, con il compito di annotare i problemi riscontrati e aggiorna il registro di revisione.

4.2. Verifica dinamica

La verifica dinamica, invece, si basa sull'esecuzione effettiva del codice per osservare il comportamento del sistema in tempo reale.
Serve a valutare la correttezza funzionale, l'affidabilità e le prestazioni dell'applicazione attraverso casi di test progettati per riprodurre situazioni d'uso reali.

Le principali attività includono:

- **Unit Testing**: verifica di singole funzioni o metodi, in particolare nelle classi del backend e nei componenti di interfaccia.
- **Integration Testing**: controllo della comunicazione tra moduli frontend e backend (es. API REST e database MongoDB).
- **System Testing**: validazione del comportamento globale del sistema su dispositivi Android reali.
- **User Acceptance Testing (UAT)**: test finali condotti da utenti esterni per verificare la soddisfazione dei requisiti funzionali e di usabilità.

Durante l'esecuzione dei test dinamici, i risultati attesi vengono confrontati con quelli ottenuti, documentando eventuali discrepanze nel registro dei test.

Questo approccio combinato permette di individuare tempestivamente errori sia logici e strutturali (tramite analisi statica) sia comportamentali e di performance (tramite analisi dinamica), assicurando una copertura completa della verifica dei requisiti.

5. Casi di test

Vengono di seguito elencati i casi di test associati ai requisiti funzionali specificati nel documento dei requisiti (*D3*).
Ogni tabella descrive la funzionalità testate, le condizioni di esecuzione, gli input previsti, gli output attesi e le post-condizioni, insieme all'esito del test.
I test del **frontend** saranno condotti da *Davide Lago*, mentre i test del **backend** da *Hamza Rofi*, con la supervisione generale di Samuele Zanutto.

Tabella d'esempio:

Funzionalità	Nome della funzionalità
RF	Riferimento alla specifica dei requisiti indicata con S-XX
Descrizione	Viene fornita una descrizione dettagliata del test che si andrà ad eseguire.
Pre Condizioni	Condizioni necessarie affinché possa essere eseguito il test.
Input	Azioni o dati in ingresso che innescano la funzionalità.
Output attesi	Insieme di output che dovrebbe fornire la funzionalità.
Post Condizioni	Condizioni che si dovrebbero riscontrare dopo l'esecuzione della funzionalità.
Esito	Esito riscontrato al completamento dell'operazione di testing della funzionalità, con data e valutatore. La data si riferisce all'ultimo test eseguito.
Priorità	Viene assegnata una priorità tra bassa, media o alta.

Casi test:

Funzionalità	Creazione profilo
RF	S-01
Descrizione	Verifica che l'utente possa creare un nuovo profilo inserendo i dati richiesti (username, email, password, ecc.) e che questi vengano correttamente registrati nel database.
Pre Condizioni	L'applicazione è installata e avviata, nessun profilo precedentemente creato.
Input	Inserimento dei dati nel form di registrazione e pressione del tasto "Crea profilo".
Output attesi	Conferma di registrazione avvenuta con successo e redirect automatico alla schermata di login.
Post Condizioni	Profilo utente correttamente salvato nel database.
Esito	Eseguito il 18/12/2025 – *Tester: Hamza Rofi* – Esito: positivo
Priorità	Alta

Funzionalità	Login
RF	S-02
Descrizione	Verifica che un utente registrato possa autenticarsi correttamente.
Pre Condizioni	Profilo già registrato nel database.
Input	Inserimento di email e password, pressione del tasto "Accedi".
Output attesi	Accesso alla home page dell'app con caricamento dei dati dell'utente.
Post Condizioni	Sessione utente attiva e autenticazione valida.
Esito	Eseguito il 18/12/2025 – *Tester: Hamza Rofi* – Esito: positivo
Priorità	Alta

Funzionalità	Visualizzazione emergenza
RF	S-03
Descrizione	Verifica che l'utente possa visualizzare nel dettaglio un'emergenza selezionata dalla mappa o da una notifica.
Pre Condizioni	Esistono emergenze registrate nel sistema.
Input	Selezione di un'emergenza sulla mappa o da notifica push.
Output attesi	Schermata di dettaglio con tipo di emergenza, localizzazione e timestamp.
Post Condizioni attese	Nessuna modifica ai dati, solo lettura.
Esito	Eseguito il 19/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Alta

Funzionalità	Visualizzazione mappa emergenze
RF	S-04
Descrizione	Verifica che la mappa mostri correttamente tutte le emergenze geolocalizzate.
Pre Condizioni	Connessione Internet attiva e geolocalizzazione abilitata.
Input	Apertura della sezione "Mappa emergenze".
Output attesi	Visualizzazione di marker sulla mappa per ogni emergenza attiva.
Post Condizioni attese	Mappa aggiornata in tempo reale.
Esito	Eseguito il 19/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Alta

Funzionalità	Creazione emergenza
RF	S-05
Descrizione	Verifica che un utente possa segnalare un'emergenza inserendo i dati richiesti e inviandoli al server.
Pre Condizioni	Utente autenticato, connessione Internet attiva.
Input	Inserimento tipo emergenza, descrizione e posizione.
Output attesi	Conferma di segnalazione inviata con successo.
Post Condizioni attese	Nuova emergenza salvata nel database e visibile sulla mappa.
Esito	Eseguito il 19/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Alta

Funzionalità	Notifica emergenza
RF	S-06
Descrizione	Verifica che l'utente riceva una notifica push in caso di nuova emergenza nella zona d'interesse.
Pre Condizioni	Notifiche attive e posizione GPS abilitata.
Input	Creazione di un'emergenza in una zona vicina all'utente.
Output attesi	Ricezione di notifica con tipo e posizione dell'emergenza.
Post Condizioni attese	L'utente può aprire la notifica e visualizzare i dettagli dell'emergenza.
Esito	Eseguito il 20/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Alta

Funzionalità	Visualizzazione meteo
RF	S-07
Descrizione	Verifica che vengano correttamente mostrati i dati meteorologici aggiornati.
Pre Condizioni	Connessione Internet attiva.
Input	Apertura sezione “Meteo”.
Output attesi	Visualizzazione di temperatura, vento e livello mare.
Post Condizioni attese	Aggiornamento automatico dei dati.
Esito	Eseguito il 20/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Media

Funzionalità	Collegamento sito Protezione Civile
RF	S-08
Descrizione	Verifica che l’app apra correttamente il collegamento al sito ufficiale della Protezione Civile.
Pre Condizioni	Connessione Internet attiva.
Input	Pressione sul link “Vai al sito Protezione Civile”.
Output attesi	Apertura browser esterno con la homepage del sito.
Post Condizioni attese	Nessuna modifica ai dati locali.
Esito	Eseguito il 20/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Bassa

Funzionalità	Chiama soccorsi
RF	S-09
Descrizione	Verifica che la funzione di chiamata d'emergenza apra correttamente il dialer del dispositivo.
Pre Condizioni	Permesso di chiamata abilitato.
Input	Pressione del pulsante "Chiama soccorsi".
Output attesi	Apertura dialer con numero preimpostato (es. 112).
Post Condizioni attese	L'utente può procedere alla chiamata.
Esito	Eseguito il 21/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Media

Funzionalità	Visualizzazione profilo
RF	S-10
Descrizione	Verifica che l'utente possa accedere alla sezione profilo e visualizzare le proprie informazioni.
Pre Condizioni	Utente autenticato.
Input	Apertura sezione "Profilo".
Output attesi	Visualizzazione dati personali e impostazioni account.
Post Condizioni attese	Nessuna modifica ai dati.
Esito	Eseguito il 21/12/2025 – *Tester: Davide Lago* – Esito: positivo
Priorità	Media

Funzionalità	Modifica profilo
RF	S-11
Descrizione	Verifica che l'utente possa modificare correttamente i propri dati personali.
Pre Condizioni	Utente autenticato.
Input	Modifica dei campi nome o email e salvataggio.
Output attesi	Messaggio di conferma e aggiornamento immediato delle informazioni.
Post Condizioni attese	Nuovi dati salvati nel database.
Esito	Eseguito il 21/12/2025 – *Tester: Hamza Rofi* – Esito: positivo
Priorità	Alta

Funzionalità	Logout
RF	S-12
Descrizione	Verifica che l'utente possa effettuare il logout dal proprio profilo e tornare alla schermata iniziale.
Pre Condizioni	Utente autenticato.
Input	Pressione del pulsante "Logout".
Output attesi	Chiusura della sessione e ritorno alla pagina di login.
Post Condizioni attese	Nessuna sessione attiva nel dispositivo.
Esito	Eseguito il 22/12/2025 – *Tester: Hamza Rofi* – Esito: positivo
Priorità	Alta

6. Schedule del testing

Questa sezione descrive la pianificazione temporale delle attività di testing, indicando per ogni data i test previsti e la relativa responsabilità.
Le prove sono state distribuite nei giorni precedenti alla consegna della **Versione 1.0 (22/12/2025)**, in modo da garantire il completamento dei test funzionali e la validazione dei requisiti non funzionali.

6.1. Pianificazione dei test

Data	Test previsti	Responsabile primario	Supervisione / supporto
18/12/2025	• Test 1 – Creazione profilo • Test 2 – Login	Hamza Rofi	Prof. Cortesi
19/12/2025	• Test 3 – Visualizzazione emergenza • Test 4 – Visualizzazione mappa emergenze • Test 5 – Creazione emergenza	Davide Lago (frontend) Hamza Rofi (backend)	Prof. Cortesi
20/12/2025	• Test 6 – Notifica emergenza • Test 7 – Visualizzazione meteo • Test 8 – Collegamento sito Protezione Civile	Davide Lago	Utente esterno (tester volontario)
21/12/2025	• Test 9 – Chiama soccorsi • Test 10 – Visualizzazione profilo • Test 11 – Modifica profilo	Davide Lago	Prof. Cortesi
22/12/2025	• Test 12 – Logout Verifica finale d'integrazione e coerenza dati tra moduli	Davide Lago & Hamza Rofi	Samuele Zanutto (revisione e validazione documentale)

6.2. Criteri di test e requisiti associati

Criterio di Test	Requisiti verificati	Responsabile primario	Responsabile secondario
Usabilità	• RNF-1 (Grafica minimale e intuitiva) • RNF-2 (Guida onboarding) • RNF-3 (Accessibilità utenti senior)	Davide Lago	Utente esterno
Sicurezza	• RNF-9 (Sicurezza) • RNF-17 (Regolamento GDPR)	Davide Lago	Prof. Cortesi
Interoperabilità	• RNF-10 (Portabilità) • RNF-14 (Database)	Hamza Rofi	Prof. Cortesi
Robustezza	• RNF-4 (Affidabilità e stabilità) • RNF-5 (Tempi di risposta brevi)	Hamza Rofi	Samuele Zanutto
Affidabilità	• RNF-7 (Installazione) • RNF-8 (Disinstallazione)	Hamza Rofi	Prof. Cortesi
Scalabilità	• RNF-14 (Database) • RNF-18 (Sostenibilità)	Hamza Rofi	Prof. Cortesi
Globalizzazione	• RNF-3 (Accessibilità utenti senior) • RNF-18 (Sostenibilità)	Davide Lago	Samuele Zanutto
Privacy	• RNF-16 (Diritto all'oblio) • RNF-17 (GDPR)	Hamza Rofi	Prof. Cortesi
Connettività	• RNF-15 (Internet) • RF-6 (Notifica emergenza) • RF-7 (Visualizzazione meteo)	Hamza Rofi	Davide Lago
Stress e volume	• RNF-4 (Affidabilità e stabilità) • RNF-5 (Tempi di risposta brevi)	Hamza Rofi	Prof. Cortesi

Ripristino di emergenza / backup	• RNF-14 (Database) • RNF-9 (Sicurezza)	Hamza Rofi	Samuele Zanutto
Test sulle unità	• Tutti i RF da 1 a 12	Hamza Rofi (backend) Davide Lago (frontend)	Samuele Zanutto
Test di integrazione	• RF-4 (Mappa emergenze) • RF-5 (Creazione emergenza) • RF-6 (Notifica emergenza)	Hamza Rofi & Davide Lago	Prof. Cortesi

7. Procedure di registrazione dei test

Tutti i test eseguiti vengono registrati, tracciati e gestiti in modo da garantire **tracciabilità**, **riproducibilità** e **responsabilità**. Ogni esito influenza il backlog e le attività di correzione: i test **positivi** non richiedono azioni correttive; i test **negativi** generano un bug/issue che segue la procedura di risoluzione descritta in 7.3.

Strumenti ufficiali:

- **Registro test & report**: Google Sheets (tabella centrale di tracciamento esecuzioni e metriche).
- **Gestione difetti / task**: GitHub Issues + Pull Requests (collegamento diretto alla PR/commit che risolve il difetto).
- **Task tracking e backlog**: GitHub Projects.
- **Comunicazione e meeting**: Discord / Google Meet / Incontri di persona.
- **CI / Test automatici**: GitHub Actions (esecuzione unit/integration test, build, coverage report).

7.1. Monitoraggio dei test

Per ogni caso di test eseguito compilare la riga corrispondente nel Google Sheet “Registro Test” con i seguenti campi minimali:

1. TestRunID (es. TR-20251218-V1.0)
2. Data esecuzione
3. Tester (Davide Lago / Hamza Rofi / Esterno)
4. RF / S-ID (es. RF-05 / S-05)
5. Ambiente (dev / staging / device model / Android versione)
6. Input usati / passi eseguiti (breve elenco puntato)
7. Output attesi (come da specifica)
8. Output riscontrati (testo libero)
9. Esito (PASS / FAIL / BLOCKED)
10. Severità (Critical / Major / Minor / Trivial)
11. Priorità per la correzione (P0 emergente / P1 alta / P2 media / P3 bassa)
12. Issue collegata (link GitHub Issue se creato)
13. Commit/PR di riferimento (link)
14. Note / allegati (log, screenshot, file di test)
15. Tempo impiegato (minuti)

Regole rapide:

- Prima di eseguire il test verificare che l'ambiente sia aggiornato alla build indicata (tag/version).
- Annotare la build nel campo TestRunID.
- Allegare sempre almeno uno screenshot o log per i FAIL. Se il test è riproducibile, documentare i passi esatti.
- I tester segnano PASS/FAIL al termine dell'esecuzione; i risultati vengono revisionati dal PM (Samuele) entro 24h.

7.2. Aggiornamento dei test

come si mantiene il registro aggiornato:

Workflow aggiornamento risultati:

1. Esecuzione test → compilazione riga nel Google Sheet.
2. Se PASS: marcare come CLOSED nella colonna Esito. Copiare parte della riga nel report sprint.
3. Se FAIL o BLOCKED: procedere alla creazione immediata di un **GitHub Issue** seguendo il template (vedi sotto), poi inserire il link dell'Issue nella riga del foglio.
4. Aggiornare stato Issue con tag sprint e assegnazione (assignee) entro 12h dal report.
5. Dopo la correzione, il tester riesegue il test (regression) e aggiorna la riga con nuovo TestRunID e data. Solo dopo regression PASS l'Issue viene chiuso.

Template minimo per GitHub Issue (obbligatorio per ogni FAIL):

- Titolo: [RF-xx] Breve descrizione errore | Severità
- Body:
    - TestRunID:
    - Ambiente:
    - Passi per riprodurre (numerati)
    - Risultato atteso
    - Risultato osservato
    - Allegati: screenshot, log, trace
    - Temporaneità / workaround (se presente)
    - Assign to: (assegnare a Hamza o Davide in base al componente)
    - Tag: bug / security / performance / regression / blocked

7.3. Risoluzione dei problemi

Sequenza operativa per un FAIL:

1. **Segnalazione (tester)**
    - Creazione GitHub Issue + inserimento riferimento nel Registro Test.
    - Taggare la severità e stimare impatto (tester suggerisce P0/P1/P2/P3).
2. **Triage (Project Manager — Samuele Zanutto, entro 12 ore)**
    - Conferma severità; assegna la risorsa competente (Hamza per backend/integrazione, Davide per frontend/UI).
    - Se Critical (P0): convocazione immediata meeting breve e notifica al docente se impatta milestone.
3. **Correzione (sviluppatore assegnato)**
    - Creare branch feature/bugfix con nome standard (bugfix/RFxx-descrizione).
    - Implementare fix, scrivere unit test se applicabile.
    - Push e aprire PR con descrizione e riferimento all'Issue.
4. **Revisione & CI**
    - PR deve passare pipeline CI (build + unit tests + lint).
    - Code review da almeno un membro del team (diverso dall'autore).
    - Merge solo dopo approvazione e CI verde.
5. **Verifica post-fix (tester originale)**
    - Rerun del test in ambiente indicato; compilare nuovo TestRunID e aggiornare Google Sheet.
    - Se PASS: chiusura Issue con commento finale e riferimento TestRunID.
    - Se FAIL: riaprire Issue o creare nuovo Issue correlato.
6. **Metriche e reportistica (PM)**
    - Aggiornamento dashboard nel Google Sheet: open defects, defect density, MTTR (Mean Time To Repair), PASS rate.
    - Revisione settimanale in sprint review.

SLA e priorità:

- P0 (Critical): risoluzione entro 48 ore; blocco release se impatta maggiori funzionalità.
- P1 (Alta): risoluzione entro 7 giorni lavorativi.
- P2 (Media): risoluzione entro il prossimo sprint.
- P3 (Bassa): risoluzione in backlog futuro, documentata.

Escalation:

- Se lo sviluppo non può rispettare SLA P0, PM notifica docente e si valuta rollback o ritardo consegna; si tiene riunione straordinaria.

7.4. Requisiti di tracciabilità, archiviazione e reporting

- Ogni esecuzione di test deve avere TestRunID univoco e link alla build/tag git.
- Conservare i log e gli allegati nell'Issue GitHub (o in cartella Google Drive con link nell'Issue).
- Report giornaliero automatico (estratto Google Sheets) da inviare al gruppo e al docente prima del meeting di fine sprint.
- Al termine di ogni ciclo di test (prima della release) produrre il **Test Summary Report** contenente: percentuale PASS, numero difetti aperti/chiusi, elenco difetti critici, raccomandazioni per rilascio.

7.5. Automazione e test continui

- Le suite di unit/integration test vengono eseguite automaticamente su ogni PR tramite **GitHub Actions**; il risultato della pipeline è requisito per la fusione.
- I test manuali e di UAT sono registrati con TestRunID separati e collegati alla build su cui sono eseguiti.

7.6. Archiviazione e retention

Registro Test (Google Sheets) e Issues GitHub sono conservati per tutta la durata del corso e almeno 12 mesi dopo la consegna finale; archiviazione finale in repository del progetto con snapshot PDF del Test Summary Report.

Checklist rapida per il tester (prima di segnare PASS/FAIL):

- Ambiente corrisponde alla build indicata?
- Passi riproducibili documentati?
- Screenshot e log allegati?
- Issue creato (se FAIL) con template compilato?
- Entry aggiornata nel Registro Test con TestRunID e link Issue/PR?

8. Requisiti hardware e software

Questa sezione descrive in dettaglio le risorse tecniche necessarie per eseguire il processo di testing e garantire il corretto funzionamento dell'applicazione, sia dal punto di vista **hardware** (dispositivi e infrastruttura di rete), sia **software** (strumenti, linguaggi, framework e servizi utilizzati).

8.1. Requisiti hardware

Per eseguire correttamente i test funzionali e non funzionali, sono necessari i seguenti dispositivi e risorse fisiche:

Categoria	Descrizione	Specifiche minime	Note
PC di sviluppo e testing	Utilizzati per lo sviluppo, il debug e la compilazione dell'applicazione.	• CPU Intel i5 / AMD Ryzen 5 o superiore • RAM 8 GBSSD 256 GB	Utilizzati da tutti i membri del team.
Smartphone Android	Test del comportamento reale dell'applicazione.	• Android 8.1 (API 27) o superiore • RAM ≥ 2 GB • Spazio libero ≥ 300 MB	Test principali su dispositivi reali (es. Xiaomi, Samsung, OnePlus).
Connessione Internet	Necessaria per accedere al database remoto e inviare/recepire segnalazioni.	• Banda minima 10 Mbps	Preferibilmente Wi-Fi stabile per evitare falsi positivi durante i test di connettività.
Server remoto (hosting MongoDB)	Utilizzato per ospitare il database e le API REST.	• Uptime ≥ 99%Connessione sicura HTTPS	Fornito da servizio cloud (MongoDB Atlas).
GPS e servizi di localizzazione	Necessari per testare la geolocalizzazione delle emergenze.	• Sensore GPS attivo e con precisione ≤ 10m	Test reali eseguiti su smartphone fisici, non emulatori.

8.2. Requisiti software

La componente software copre sia l'ambiente di sviluppo che gli strumenti dedicati al testing e alla gestione del progetto.

Categoria	Software / Servizio	Versione minima / specifica	Scopo
Sistema operativo sviluppo	Windows 10 / Ubuntu 22.04 LTS	Aggiornato all'ultima versione	Compatibilità con IDE e strumenti CLI.
IDE frontend	Visual Studio Code	1.85 o superiore	Sviluppo e debugging dell'app Angular.
IDE mobile	Android Studio	Jellyfish o superiore	Packaging e testing dell'app su dispositivo/emulatore Android.
Database	MongoDB Atlas	Cloud 5.0+	Gestione segnalazioni e utenti.
Linguaggi e framework	JavaScript / TypeScript, Angular, Node.js	Angular 17, Node.js 20	Frontend e backend del sistema.
Versionamento	Git + GitHub	n.d.	Controllo versione e collaborazione.
Strumenti di testing	Postman, Jest, Cypress	Postman 10+, Jest 29+, Cypress 13+	Test API, unit test e end-to-end testing.
Gestione documentazione	Google Drive, Microsoft Word / PDF	Ultima versione	Redazione e aggiornamento deliverable.
CI/CD	GitHub Actions	n.d.	Esecuzione automatica dei test e build.
Browser di test	Google Chrome / Mozilla Firefox	Ultima versione	Testing responsive e visualizzazione mappe.
Strumenti di comunicazione	Discord, Google Meet	n.d.	Coordinamento e review di gruppo.

8.3. Configurazione dell'ambiente di Test

- Tutti i test verranno condotti inizialmente in ambiente locale su macchina personale e successivamente replicati in ambiente staging con connessione al database remoto.
- I test di integrazione e regressione saranno eseguiti tramite pipeline CI configurata su GitHub Actions.
- I test mobile verranno condotti sia su emulatori Android Studio che su dispositivi fisici reali, al fine di verificare performance, usabilità e compatibilità.

8.4. Compatibilità e Scalabilità

sistema dovrà garantire compatibilità con:

- Dispositivi **Android 8.1 (API 27)** o superiori.
- Browser **Chrome, Edge, Firefox** aggiornati per la versione web.
- Risoluzioni schermo comprese tra **720x1280** e **1440x3120** pixel.

La soluzione è progettata per supportare fino a **1.000 utenti simultanei** in ambiente cloud MongoDB, con possibilità di scalabilità orizzontale tramite configurazione **cluster**.

8.5. Considerazioni di sicurezza e sostenibilità

- Tutti gli ambienti di test saranno configurati in HTTPS con certificato SSL valido.
- I dati generati durante il testing verranno periodicamente eliminati per evitare sovraccarico del database e rispettare il principio di **minimizzazione GDPR**.
- Verrà utilizzato **hardware già disponibile** all'interno del team, minimizzando l'impatto economico e ambientale del processo di testing.

9. Vincoli che condizionano il testing

La fase di testing può essere influenzata da diversi vincoli di natura tecnica, organizzativa o temporale, che potrebbero incidere sull'efficacia e sulla puntualità dell'intero processo di verifica del sistema. Di seguito vengono elencati i principali vincoli identificati e le relative considerazioni preventive.

9.1. Vincoli tecnici

- **Limitazioni hardware**: alcuni dispositivi Android di fascia bassa possono non rispettare i requisiti minimi (RAM, CPU o spazio di archiviazione), causando rallentamenti o crash durante l'esecuzione dei test. È previsto l'uso di almeno un dispositivo fisico e un emulatore Android per garantire la copertura dei test su più configurazioni.
- **Infrastruttura cloud e connessione Internet**: il sistema dipende dal corretto funzionamento dei servizi esterni (MongoDB Atlas, API meteo, API geolocalizzazione). Problemi di rete o downtime del provider possono impedire i test di integrazione. È consigliato pianificare finestre di testing ridondanti e disporre di un database locale di emergenza.
- **Configurazioni software non uniformi**: differenze tra ambienti di sviluppo (versioni di Node.js, Angular, dipendenze NPM) possono generare comportamenti incoerenti. Ogni test verrà eseguito in un ambiente configurato tramite file.env condiviso e versionato su GitHub

9.2. Vincoli organizzativi

- **Disponibilità dei membri del team**: i componenti responsabili del testing (Davide Lago per il frontend, Hamza Rofi per il backend) devono conciliare la fase di test con altre attività accademiche e progettuali. Un'assenza prolungata può ritardare l'intera catena di validazione. Viene quindi mantenuto un calendario condiviso con l'assegnazione di backup temporanei.
- **Mancanza di competenze specifiche**: alcune aree di test (ad esempio, stress testing e test di integrazione CI/CD) richiedono conoscenze avanzate non ancora consolidate nel gruppo. Si prevedono

sessioni di autoformazione e consultazione di risorse ufficiali (documentazione Angular, Node.js, MongoDB Atlas).

- **Coordinamento e comunicazione**: eventuali ritardi di sincronizzazione tra frontend e backend possono rendere non riproducibili determinati casi di test. Si useranno canali dedicati Discord e GitHub Issues per segnalare tempestivamente le anomalie.

9.3. Vincoli temporali

- **Scadenze accademiche**: le date di testing devono rispettare la schedule riportata nel capitolo 6 e terminare prima della consegna ufficiale del 22/12/2025. Ogni ritardo nella fase di sviluppo impatterà direttamente sul tempo utile per l'esecuzione e la validazione dei test.
- **Risoluzione dei bug**: la correzione degli errori emersi durante il testing può richiedere tempi imprevisti. È quindi riservato un margine temporale del 10–15% rispetto al calendario pianificato per garantire flessibilità nelle iterazioni correttive.
- **Dipendenza da strumenti esterni**: aggiornamenti automatici di framework o librerie (es. Angular, Node.js, Firebase SDK) durante la finestra di testing possono alterare il comportamento del software e richiedere adattamenti urgenti.

9.4. Vincoli ambientali e logistici

- **Connessione e copertura GPS**: i test basati su geolocalizzazione o mappe dipendono dalla qualità del segnale GPS e della rete dati. Zone con scarsa copertura potrebbero fornire risultati falsati o ritardati.
- **Disponibilità fisica dei dispositivi**: un guasto o la perdita di un dispositivo di test comporterebbe la sospensione dei test reali su hardware mobile. Per mitigare il rischio, è previsto l'uso di emulatori Android come soluzione temporanea.

9.5. Vincoli normativi e di sicurezza

- **Trattamento dei dati personali**: i test dovranno rispettare le normative **GDPR** e garantire l'anonimato dei dati simulati. Nessuna informazione reale verrà memorizzata o condivisa.
- **Accesso ai servizi API esterni**: alcune API (come quelle meteo) impongono limiti di richieste per account gratuito, influenzando i test di carico e stress. Verranno adottate chiavi API di backup e caching locale per ridurre le chiamate ripetute.

10. Riferimenti

Le risorse utilizzate per la stesura di questo documento sono:

- Slides e risorse scaricate dalla piattaforma moodle del corso.
- Documento Piano di Progetto (D2).
- Documento dei requisiti (D3).